

MQTT-Schnittstelle

MQTT-Schnittstelle der OpenMower TAF App

Diese Seite beschreibt die MQTT-Schnittstelle, die von der OpenMower TAF App verwendet wird. Sie ist als Arbeits- und Integrationsdokumentation gedacht: Man sieht, welche Topics die App abonniert, auf welche Topics sie schreibt, welche Payloads erwartet werden und welche Teile aus Kompatibilitätsgründen noch unterstützt werden.

Versionsbezug

Punkt	Stand
Dokumentversion	MQTT-API-Doku 1.0
App	OpenMower-TAF-App 1.0.0+1
ROS / Backend	OpenMower mit ROS Noetic / ROS 1
Doku-Stand	08.06.2026
Quelle	aktuelles App-Paket, Datei <code>lib/io/mqtt_connection.dart</code> und Controller unter <code>lib/controllers/</code>

Wenn App oder Backend in einem anderen Release-Stand betrieben werden, können einzelne Topics oder Payload-Felder abweichen. Die App ist JSON-first ausgelegt, akzeptiert an einigen Stellen aber weiterhin BSON- oder Legacy-Topics.

Build-Log-Hinweis

Die aktuellen Build-Logs wurden berücksichtigt. Der Android-Build brach wegen `:app:checkDebugAarMetadata` ab, weil `androidx.browser:browser:1.9.0`, `androidx.core:core-ktx:1.17.0` und `androidx.core:core:1.17.0` mindestens Android Gradle Plugin 8.9.1 verlangen, während das Projekt 8.7.3 verwendete. Das Paket wurde deshalb auf Android Gradle Plugin 8.11.1, Kotlin Gradle Plugin 2.2.20 und Gradle Wrapper 8.14.3 angehoben. Die vorhandene Docker/Web-Build-Korrektur für `flutter_launcher_icons_web.yaml` bleibt bestehen.

Grundprinzip

Die App verbindet sich per MQTT mit dem OpenMower-Backend. Im Web-Release nutzt sie standardmäßig WebSocket-MQTT über den Host der geöffneten Seite und Port 9001. In der Android-/Desktop-Variante nutzt sie die in den App-Einstellungen gesetzten MQTT-Daten. Der MQTT-Client meldet sich mit einer Client-ID im Format `om-client-⟨zufallszahl⟩` an.

Die App verwendet unterschiedliche QoS-Stufen:

- Status- und Diagnosedaten meist `atMostOnce` oder `atLeastOnce`.
- Konfigurationsänderungen, Karten-Speichern und Actions meist `exactlyOnce`.
- JSON-Payloads werden als UTF-8-codiertes JSON übertragen.
- BSON-Payloads werden mit BSON serialisiert und dienen vor allem der Kompatibilität.
- Viele Status-Payloads dürfen entweder direkt als Objekt kommen oder unter `d` gekapselt sein.

Topic-Übersicht

Von der App abonnierte Topics

Topic	Format	QoS	Zweck
actions/bson	BSON	exactlyOnce	Liste verfügbarer Aktionen mit action_id und enabled .
robot_state/json	JSON	atMostOnce	Hauptzustand des Mähers.
robot_state/bson	BSON	atMostOnce	Hauptzustand des Mähers, BSON-kompatibel.
robot_pose/json	JSON	atMostOnce	Positionsdaten des Mähers.
sensors/pose/json	JSON	atMostOnce	Alternative Pose-Quelle.
sensors/status/json	JSON	atMostOnce	Allgemeine Sensor-/Statusdaten.
sensors/+/json	JSON	atMostOnce	Einzelne JSON-Sensorwerte.
sensors/+/bson	BSON	atMostOnce	Einzelne BSON-Sensorwerte.
sensor_infos/bson	BSON	atLeastOnce	Metadaten zu Sensoren.
version	BSON	atLeastOnce	Backend-/Softwareversion.
map/json	JSON	atLeastOnce	Karte und Flächen.
map/bson	BSON	atLeastOnce	Karte und Flächen, BSON-kompatibel.
map/validation/json	JSON	atLeastOnce	Validierungsantwort für Kartendaten.
map/validation/bson	BSON	atLeastOnce	Validierungsantwort, BSON-kompatibel.
map/overlay/json	JSON	atMostOnce	Zusätzliche Linien/Polygone für die Karte.
map/overlay/bson	BSON	atMostOnce	Overlay, BSON-kompatibel.
map_overlay/json	JSON	atMostOnce	Legacy-Overlay-Topic.
map_overlay/bson	BSON	atMostOnce	Legacy-Overlay-Topic, BSON.
map/mowing_progress/json	JSON	atMostOnce	Mähfortschritt inklusive Pfaden.
map/mowing_progress/status/json	JSON	atMostOnce	Mähfortschritt als Status-/Kurzmeldung.
timetable/json	JSON	atLeastOnce	Zeitplan.
timetable/bson	BSON	atLeastOnce	Zeitplan, BSON-kompatibel.
timetable/validation/json	JSON	atLeastOnce	Validierungsantwort für Zeitplan.
timetable/validation/bson	BSON	atLeastOnce	Validierungsantwort, BSON-kompatibel.
/openmower/timetable/status/json	JSON	atLeastOnce	Legacy-Zeitplanstatus.
timetable/response/json	JSON	atLeastOnce	Legacy-Antwort.
timetable/ack/json	JSON	atLeastOnce	Legacy-ACK.
/openmower/timetable/action_result/json	JSON	atLeastOnce	Legacy-Action-Ergebnis.
/openmower/time/status/json	JSON	atLeastOnce	Zeitservice-Status.
/openmower/time/status/bson	BSON	atLeastOnce	Zeitservice-Status, BSON.
/openmower/time/action_result/json	JSON	atLeastOnce	Antwort des Zeitservice.
/openmower/time/action_result/bson	BSON	atLeastOnce	Antwort des Zeitservice, BSON.

Topic	Format	QoS	Zweck
/openmower/time/config/status/json	JSON	atLeastOnce	Zeitservice-Konfigurationsstatus.
/openmower/time/config/status/bson	BSON	atLeastOnce	Zeitservice-Konfigurationsstatus, BSON.
statustransition_log/json	JSON	atLeastOnce	Statuswechsel-Protokoll.
settings/mower_logic/json	JSON	atLeastOnce	Software-/Mäher-Logik-Einstellungen.
settings/mower_logic/validation/json	JSON	atLeastOnce	Validierung der Software-Einstellungen.
settings/ll_board/json	JSON	atLeastOnce	Hardware-/Low-Level-Board-Einstellungen.
settings/ll_board/validation/json	JSON	atLeastOnce	Validierung der Hardware-Einstellungen.

Von der App veröffentlichte Topics

Topic	Format	QoS	Zweck
action	Text	exactlyOnce	Führt eine Action aus; Payload ist die Action-ID als Text.
teleop	BSON	atMostOnce/atLeastOnce	Joystick-Steuerung mit vx und vz.
map/set/renew/json	JSON	atLeastOnce	Karte neu vom Backend anfordern.
map/set/json	JSON	exactlyOnce	Karte/Flächen speichern.
timetable/set/renew/json	JSON	atLeastOnce	Zeitplan neu anfordern.
timetable/set/json	JSON	exactlyOnce	Zeitplan speichern.
timetable/set/suspension/json	JSON	exactlyOnce	AutoMow-Aussetzung setzen.
/openmower/time/action/json	JSON	atLeastOnce	Zeitservice-Aktionen.
/openmower/time/config/set/json	JSON	exactlyOnce	Zeitservice-Konfiguration speichern.
statustransition_log/set/renew/json	JSON	atLeastOnce	Statusprotokoll anfordern.
settings/mower_logic/set/renew/json	JSON	atLeastOnce	Software-Einstellungen neu anfordern.
settings/mower_logic/set/session/json	JSON	exactlyOnce	Softwarewerte live für die Session setzen.
settings/mower_logic/set/persistent/json	JSON	exactlyOnce	Softwarewerte dauerhaft speichern.
settings/ll_board/set/renew/json	JSON	atLeastOnce	Hardwarewerte neu anfordern.
settings/ll_board/set/session/json	JSON	exactlyOnce	Hardwarewerte live für die Session setzen.
settings/ll_board/set/persistent/json	JSON	exactlyOnce	Hardwarewerte dauerhaft speichern.

Actions

Die App liest die verfügbaren Actions aus `actions/bson`. Erwartet wird ein BSON-Objekt mit einer Liste unter `d`.

```
{
  "d": [
```

```

{ "action_id": "mower_logic:idle/start_mowing", "enabled": 1 },
{ "action_id": "mower_logic:mowing/pause", "enabled": 1 },
{ "action_id": "mower_logic:mowing/skip_area", "enabled": 1 }
]
}

```

Nur Actions mit `enabled > 0` werden in der App als verfügbar betrachtet. Beim Drücken einer Aktion veröffentlicht die App die Action-ID als reinen Text auf `action`.

Wichtige von der UI verwendete Action-IDs:

Action-ID	Verwendung in der App
<code>mower_logic:idle/start_mowing</code>	Mähbetrieb starten.
<code>mower_logic:mowing/pause</code>	Laufenden Mähbetrieb pausieren.
<code>mower_logic:mowing/abort_mowing</code>	Laufenden Mähbetrieb abbrechen.
<code>mower_logic:mowing/skip_area</code>	Aktuelle Fläche überspringen.
<code>mower_logic:mowing/skip_path</code>	Aktuellen Pfad überspringen.
<code>mower_logic:idle/start_area_recording</code>	Flächenaufzeichnung starten.
<code>mower_logic:area_recording/start_recording</code>	Aufzeichnung aktivieren.
<code>mower_logic:area_recording/stop_recording</code>	Aufzeichnung stoppen.
<code>mower_logic:area_recording/finish_mowing_area</code>	Aufzeichnung als Mähfläche speichern.
<code>mower_logic:area_recording/finish_navigation_area</code>	Aufzeichnung als Navigationsfläche speichern.
<code>mower_logic:area_recording/finish_discard</code>	Aufzeichnung verwerfen.
<code>mower_logic:area_recording/record_dock</code>	Docking-Position aufnehmen.
<code>mower_logic:area_recording/exit_recording_mode</code>	Aufzeichnungsmodus verlassen.
<code>mower_logic:area_recording/auto_point_collecting_enable</code>	Automatisches Punktsammeln aktivieren.
<code>mower_logic:area_recording/auto_point_collecting_disable</code>	Automatisches Punktsammeln deaktivieren.
<code>mower_logic:area_recording/collect_point</code>	Einzelnen Punkt aufnehmen.

Robot State und Sensoren

`robot_state/json` und `robot_state/bson`

Der Robot State ist die wichtigste Statusquelle. JSON kann direkt oder unter `d` gekapselt sein.

```

{
  "d": {
    "pose": {
      "x": 1.2,
      "y": -3.4,
      "heading": 0.52,
      "pos_accuracy": 0.03,
      "heading_accuracy": 0.1,
      "heading_valid": 1
    },
    "current_state": "MOWING",
    "current_sub_state": "PATH_FOLLOWING",
    "emergency": 0,
    "is_charging": 0,
    "rain_detected": 0,
    "gps_percentage": 98,
    "battery_percentage": 76,
    "load_factor_computed": 1.12,
    "load_factor_effective": 1.05,
    "current_area": 2,
    "current_area_id": "area-02",
    "current_path": 4,
    "current_path_index": 4,
    "AutoMowSuspension": 0,
    "AutoMowID": "weekday-evening"
  }
}

```

Die App verwendet unter anderem diese Felder:

Feld	Bedeutung
<code>pose.x</code> , <code>pose.y</code> , <code>pose.heading</code>	Position und Ausrichtung. Die App spiegelt die Y-Achse für die Darstellung.
<code>pose.pos_accuracy</code> , <code>pose.heading_accuracy</code> , <code>pose.heading_valid</code>	GPS-/Heading-Qualität.
<code>current_state</code> , <code>current_sub_state</code>	Haupt- und Unterzustand.
<code>emergency</code>	Not-Aus / Emergency-Status.
<code>is_charging</code>	Ladezustand.
<code>rain_detected</code>	Regen erkannt.
<code>gps_percentage</code> , <code>battery_percentage</code>	GPS- und Akkuanzeige.
<code>load_factor_computed</code> , <code>load_factor_effective</code>	Mäh-Lastfaktor.
<code>current_area</code> , <code>current_area_id</code>	Aktuelle Fläche.
<code>current_path</code> , <code>current_path_index</code>	Aktueller Pfad.
<code>AutoMowSuspension</code>	Aussetzung des automatischen Mähens. 0 bedeutet nicht ausgesetzt.
<code>AutoMowID</code>	ID des aktiven Timetable-/AutoMow-Eintrags.

Pose- und Sensor-Topics

`robot_pose/json` und `sensors/pose/json` dürfen entweder eine Pose direkt oder eine Pose unter `pose` enthalten. `sensors/status/json` darf allgemeine Robot-State-Felder enthalten und wird in den Robot State übernommen.

Einzelwerte auf `sensors/+/json` werden für bekannte Sensor-IDs direkt interpretiert. Unterstützt sind unter anderem:

- `battery`, `battery_percentage`
- `gps`, `gps_percentage`
- `emergency`
- `charging`, `is_charging`
- `rain`, `rain_detected`
- `pos_accuracy`
- `heading_accuracy`
- `heading_valid`
- `x`, `y`, `heading`

Beispiel:

42.5

oder:

```
{ "value": 42.5 }
```

`sensor_infos/bson` und `sensors/+/bson`

`sensor_infos/bson` beschreibt dynamische Sensoren. Die App unterstützt hier aktuell DOUBLE-Sensoren.

```
{
  "d": [
    {
      "sensor_id": "battery_voltage",
      "sensor_name": "V Battery",
      "unit": "V",
      "value_type": "DOUBLE",
      "min_value": 0,
      "max_value": 30,
      "has_min_max": 1,
      "lower_critical_value": 18,
      "has_critical_low": 1,
      "upper_critical_value": 29,
      "has_critical_high": 1
    }
  ]
}
```

Der eigentliche Wert kommt anschließend auf `sensors/<sensor_id>/bson`, ebenfalls unter `d`.

Karte, Flächen und Editor

map/json und map/bson

Die moderne Kartenstruktur enthält `areas` und optional `docking_stations`. Die App akzeptiert die Struktur direkt oder unter `d`.

```
{
  "d": {
    "areas": [
      {
        "id": "mow-01",
        "outline": [
          { "x": 0.0, "y": 0.0 },
          { "x": 5.0, "y": 0.0 },
          { "x": 5.0, "y": 4.0 },
          { "x": 0.0, "y": 4.0 }
        ],
        "properties": {
          "type": "mow",
          "mowing_enabled": true,
          "mowing_order": 1
        }
      }
    ],
    "docking_stations": [
      { "position": { "x": 0.5, "y": 0.5 }, "heading": 1.57 }
    ]
  }
}
```

Bereich	Erwartete Felder
<code>areas[].id</code>	Stabile Flächen-ID.
<code>areas[].outline[]</code>	Punkte mit x und y.
<code>areas[].properties.type</code>	<code>mow</code> , <code>nav</code> oder <code>obstacle</code> .
<code>areas[].properties.mowing_enabled</code>	Ob die Mähfläche aktiv ist. Standard: aktiv.
<code>areas[].properties.mowing_order</code>	Reihenfolge der Mähflächen.
<code>docking_stations[0].position</code>	Docking-Position.
<code>docking_stations[0].heading</code>	Docking-Ausrichtung.

Die App unterstützt zusätzlich ein Legacy-Format mit `meta`, `docking_pose`, `working_areas` und `navigation_areas`.

Karte anfordern und speichern

Karte neu anfordern:

```
{}
```

Topic: `map/set/renew/json`

Karte speichern:

Topic: `map/set/json`

Payload ist die aktuelle Karten-/Flächenstruktur aus der App.

Validierungsantwort:

```
{
  "valid": true,
  "remarks": ["Map gespeichert"],
  "accepted": { "areas": true }
}
```

Die App akzeptiert auch Varianten mit `ok`, `success`, `accepted`, `status` oder `result`.

Overlay

map/overlay/json, map/overlay/bson, map_overlay/json und map_overlay/bson zeigen zusätzliche Linien/Polygone in der Karte.

```
{
  "d": {
    "polygons": [
      {
        "poly": [ { "x": 0, "y": 0 }, { "x": 1, "y": 1 } ],
        "is_closed": 0,
        "line_width": 0.05,
        "color": "#ff0000"
      }
    ]
  }
}
```

Mähfortschritt

map/mowing_progress/json enthält pro Fläche den Fortschritt, geplante Pfade und gemähte Pfade. map/mowing_progress/status kann eine reduzierte Statusversion liefern; fehlende Pfadlisten bleiben dann aus dem vorherigen Zustand erhalten.

```
{
  "d": {
    "current_area_id": "mow-01",
    "areas": {
      "mow-01": {
        "area_id": "mow-01",
        "percent": 47.5,
        "state": "mowing",
        "current_path": 3,
        "current_path_id": "path-003",
        "current_path_index": 3,
        "planned_paths": [
          {
            "index": 3,
            "path_id": "path-003",
            "completed_percent": 20,
            "points": [ { "x": 1, "y": 1 }, { "x": 4, "y": 1 } ]
          }
        ],
        "mowed_paths": []
      }
    }
  }
}
```

Timetable und Zeitservice

timetable/json und timetable/bson

Der Zeitplan kann direkt oder unter d kommen. Die App normalisiert die Daten für die UI. Sichtbare Kernfelder pro Eintrag sind:

- day
- start
- end
- end_behavior
- enabled
- optional fields, auto_start, required_battery_state und eine interne ID

Beispiel:

```
{
  "d": {
    "entries": [
```

```

{
  "id": "weekday-evening",
  "day": "Monday",
  "start": "18:00",
  "end": "21:00",
  "end_behavior": "return_to_dock",
  "enabled": true,
  "fields": ["mow-01", "mow-02"]
}
],
"time_settings": {
  "timezone": "Europe/Berlin",
  "active_source": "ntp",
  "ntp_server": "pool.ntp.org"
}
}
}

```

Zeitplan neu anfordern:

```
{}
```

Topic: timetable/set/renew/json

Zeitplan speichern:

Topic: timetable/set/json

AutoMow-Aussetzung setzen:

```
{ "AutoMowSuspension": "9999-12-31T23:59:59Z" }
```

Topic: timetable/set/suspension/json

Werte für AutoMowSuspension:

Wert	Bedeutung
0	keine Aussetzung
ISO-Datum/Zeit	Aussetzung bis zu diesem Zeitpunkt
9999-12-31T23:59:59Z	unbestimmt aussetzen

Zeitservice

Die App veröffentlicht Zeitaktionen auf /openmower/time/action/json.

```
{ "request_id": "time_req-...", "action": "get_status" }
```

Weitere Actions:

Action	Zusätzliche Felder
get_status	keine
resync	preferred_source, z. B. ntp, gps, system
set_timezone	timezone
set_manual_time	timezone, local_time
set_ntp_server	ntp_server
clear_manual_time	keine

Zeitkonfiguration wird auf /openmower/time/config/set/json gespeichert:

```

{
  "time": {
    "timezone": "Europe/Berlin",
    "active_source": "ntp",
    "ntp_server": "pool.ntp.org"
  }
}

```


Statuswechsel-Protokoll

Die App fordert Protokolldaten über `statustransition_log/set/renew/json` an.

```
{ "limit": 20 }
```

Das Limit wird appseitig auf 1 bis 300 begrenzt.

Die Antwort auf `statustransition_log/json` kann direkt oder unter `d` gekapselt sein. Typisch ist eine Liste von Einträgen mit Statuswechsel, Zeitstempel und Kontextdaten.

```
{
  "d": {
    "entries": [
      {
        "from": "MOWING",
        "to": "DOCKING",
        "timestamp": "2026-06-08T08:00:00Z",
        "context": {
          "battery_percentage": 22,
          "gps_percentage": 95,
          "emergency": 0,
          "current_area_id": "mow-01"
        }
      }
    ],
    "total": 1,
    "limit": 20
  }
}
```

Die UI ist tolerant gegenüber leicht anderen Feldnamen und zeigt Rohdaten zusätzlich in Detailbereichen an.

Software-Einstellungen: `settings/mower_logic`

Status

Die App erwartet die Softwareparameter auf `settings/mower_logic/json`. Die Daten können direkt oder unter `d` gekapselt sein. Entscheidend ist ein Objekt `settings`.

```
{
  "d": {
    "settings": {
      "mow_load_factor_enabled": {
        "value": true,
        "active": true,
        "persistent": true,
        "default": false,
        "label": "Mäh-Lastregelung aktiv",
        "description": "Aktiviert die Lastregelung.",
        "type": "bool",
        "group": "Load_Factor",
        "unit": "",
        "min": null,
        "max": null,
        "expert": false,
        "session_apply_supported": true
      }
    }
  }
}
```

Die App zeigt die Einträge gruppiert an. Die Mäh-Lastregelung ist kein eigener MQTT-Namespace mehr, sondern wird aus `settings/mower_logic/json` gefiltert und in einem eigenen UI-Abschnitt dargestellt.

Renew, Session und Persistent

Neu anfordern:

```
{}
```

Topic: settings/mower_logic/set/renew/json

Live-/Sessionwert setzen:

```
{
  "mow_load_factor_enabled": { "value": true }
}
```

Topic: settings/mower_logic/set/session/json

Dauerhaft speichern:

```
{
  "mow_load_factor_enabled": {
    "value": true,
    "group": "Load_Factor",
    "expert": false
  }
}
```

Topic: settings/mower_logic/set/persistent/json

Bei Session-Schreibvorgängen sendet die App nur Werte, deren `session_apply_supported` wahr ist. Metadaten wie `group` und `expert` werden nur dauerhaft gespeichert.

Validierung

Validierungsantworten kommen auf settings/mower_logic/validation/json.

```
{
  "valid": true,
  "mode": "persistent",
  "accepted": {
    "mow_load_factor_enabled": ["value", "group", "expert"]
  },
  "rejected": {},
  "remarks": ["Gespeichert"]
}
```

Die App akzeptiert auch `applied` statt `accepted`. Ablehnungen können unter `rejected` und zusätzliche Texte unter `remarks` stehen.

Hardware-Einstellungen: settings/ll_board

Status

Low-Level-Board-Parameter kommen auf settings/ll_board/json. Die App erwartet Zahlenwerte mit Metadaten.

```
{
  "d": {
    "settings": {
      "battery_empty_voltage": {
        "value": 20.0,
        "active": 20.0,
        "persistent": 20.0,
        "default": 20.0,
        "label": "Akku leer",
        "description": "Spannung, ab der der Akku als leer gilt.",
        "unit": "V",
        "min": 0,
        "max": 30,
        "group": "Battery",
        "expert": false
      }
    }
  }
}
```

Für ältere Backends gibt es begrenzte Kompatibilität mit einfachen Low-Level-/Power-Payloads, sofern daraus Zahlenwerte ableitbar sind.

Renew, Session und Persistent

Neu anfordern:

```
{}
```

Topic: `settings/ll_board/set/renew/json`

Sessionwert setzen:

```
{
  "battery_empty_voltage": { "value": 20.0 }
}
```

Topic: `settings/ll_board/set/session/json`

Dauerhaft speichern:

```
{
  "battery_empty_voltage": {
    "value": 20.0,
    "group": "Battery",
    "expert": false
  }
}
```

Topic: `settings/ll_board/set/persistent/json`

Validierungsantworten kommen auf `settings/ll_board/validation/json` und folgen demselben Muster wie bei `settings/mower_logic/validation/json`.

Teleop / Joystick

Die Joystick-Steuerung veröffentlicht BSON auf `teleop`.

```
{
  "vx": 0.2,
  "vz": -0.1
}
```

Feld	Bedeutung
<code>vx</code>	lineare Geschwindigkeit / Vorwärtsanteil
<code>vz</code>	Drehanteil

Die QoS-Stufe hängt davon ab, ob die UI high-QoS aktiviert: `atLeastOnce` bei high-QoS, sonst `atMostOnce`.

Version

Das Topic `version` wird als BSON gelesen. Erwartet wird ein Feld `version`.

```
{ "version": "open_mower_ros ..." }
```

Der Wert wird unten im App-Menü angezeigt.

Kompatibilität und entfallene Topics

Folgende Punkte sind bewusst kompatibel oder historisch:

- JSON ist bevorzugt; BSON wird bei Robot State, Karte, Timetable, Sensoren, Overlay und Zeitstatus weiter akzeptiert.
- `map_overlay/json` und `map_overlay/bson` sind Legacy-Topics. Modern sind `map/overlay/json` und `map/overlay/bson`.
- `/openmower/timetable/...` und `/openmower/time/...` sind Legacy- bzw. Zusatz-Topics für Zeit-/Timetable-Services.
- `settings/mow_load_factor/...` wird nicht mehr aktiv abonniert. Die Mäh-Lastregelung kommt aus `settings/mower_logic/json`.

Empfehlungen für Backend-Implementierungen

1. Neue Funktionen sollten JSON-first umgesetzt werden.
2. Status-Payloads sollten entweder direkt als Objekt oder sauber unter `d` gekapselt veröffentlicht werden.
3. Schreibbefehle sollten eine Validierungsantwort auf dem passenden `validation/json`-Topic liefern.
4. `accepted/applied`, `rejected`, `remarks`, `valid`, `ok`, `success`, `status` und `result` sollten konsistent verwendet werden.
5. Retained Status-Topics sind sinnvoll, damit die App nach Verbindungsaufbau sofort Daten sieht.
6. Actions sollten nur mit `enabled > 0` ausgeliefert werden, wenn sie im aktuellen Zustand wirklich ausführbar sind.
7. Für Flächen sollte eine stabile `id` vergeben werden, damit Mähfortschritt, Skip-Funktionen und Anzeige zuverlässig zuordenbar bleiben.

Beispiel-Test mit `mosquitto_pub`

Karte anfordern:

```
mosquitto_pub -h <host> -t 'map/set/renew/json' -m '{}'
```

Software-Einstellungen anfordern:

```
mosquitto_pub -h <host> -t 'settings/mower_logic/set/renew/json' -m '{}'
```

Action starten:

```
mosquitto_pub -h <host> -t 'action' -m 'mower_logic:idle/start_mowing'
```

Robot State testweise veröffentlichen:

```
mosquitto_pub -h <host> -t 'robot_state/json' -m '{"d":{"current_state":"IDLE","battery_percentage":75,"gp
```

[Zurück zur Dokumentations-Startseite](#) · [PDF-Version herunterladen](#)